Scaling Stateful Services

partition the state, give each piece one owner, route to it

Watch the 4-min video

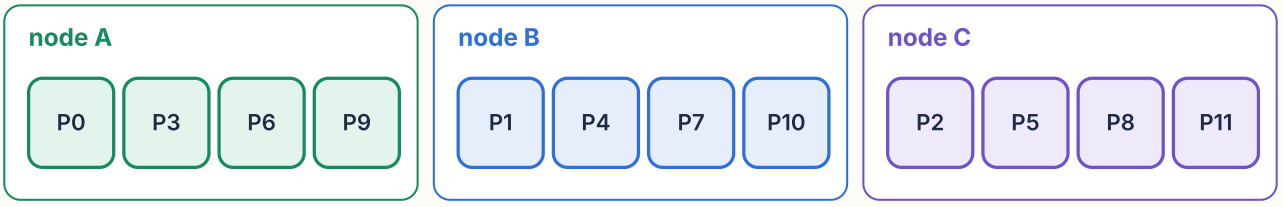
In one line: to scale a service that must keep state in memory, partition the state by key, give each partition a single owner (consistent hashing or a coordinator), route requests to the owner, and handle rebalancing carefully by rebuilding, transferring or reconnecting, while keeping the state recoverable from a durable log.

1 Some state must stay in memory

Not everything can be stateless. A chat server holds a million open WebSocket connections. A game server keeps the live state of each match in memory. A stream processor keeps running counts per user. That state must live in memory, close to the code, for speed. How do you scale it across many machines?

2 Ownership

Split the state into **partitions** by a key: user ID, room ID, match ID. Each partition is **owned by exactly one node** at a time. Requests for a key must reach the node that owns it. With single ownership there are no conflicting copies, and the owner can keep everything in memory and process its keys one at a time, without locks.



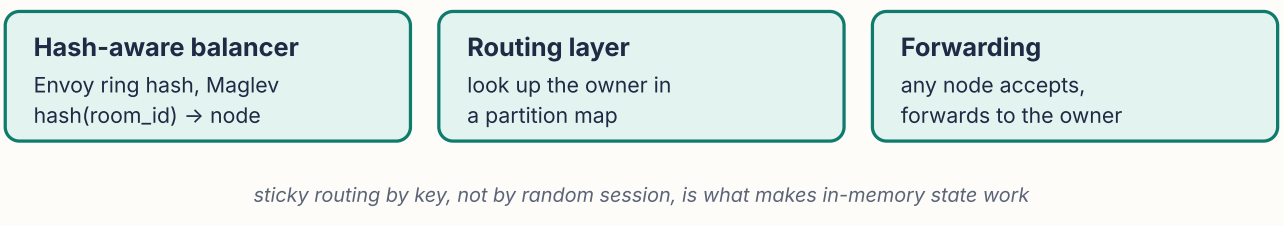
each partition (rooms, users, matches) has exactly one owner at a time; requests for its keys must reach that owner

Figure 1. Twelve partitions spread over three nodes, each with exactly one owner.

3 Deciding who owns what, and routing

Consistent hashing (lesson 5) maps keys to nodes, so adding a node moves only a small share. Or a **coordinator** (ZooKeeper, etcd, a Kafka consumer group) assigns partitions to nodes and keeps a shared map. Either way, every router must agree on the current owner.

Then requests must reach the owner: a hash-aware load balancer (Envoy ring hash, Maglev), a routing layer that looks up the partition map, or any node that accepts and forwards.



sticky routing by key, not by random session, is what makes in-memory state work

Figure 2. Three ways to get each request to the node that owns its key.

4 The hard part: rebalancing

When a node joins, leaves or crashes, partitions move, and the new owner needs the state. Options:

- **Rebuild** it from a durable source: a database, or by replaying a log (Kafka Streams restores state from changelog topics).
- **Transfer** it from the old owner before switching (graceful moves).
- **Reconnect**: clients re-establish it (WebSocket clients reconnect and resubscribe).

During the move, **pause or buffer** requests for that partition, so two nodes never both think they own it. Leases or **fencing tokens** (an epoch number that increases with each new owner, checked by storage) stop an old owner that doesn't know it was replaced.

WORKED EXAMPLE: A WEBSOCKET CHAT GATEWAY

Each gateway node owns the connections of the users connected to it. A registry in Redis maps `user:bob → gateway 2` (with a TTL refreshed by heartbeats). When a message arrives for Bob, the chat service looks up the owning node and sends the message there, which pushes it down the open socket. If a node dies, its users reconnect, land on other nodes, and the registry updates; messages sent meanwhile are kept in the chat store and delivered on reconnect.

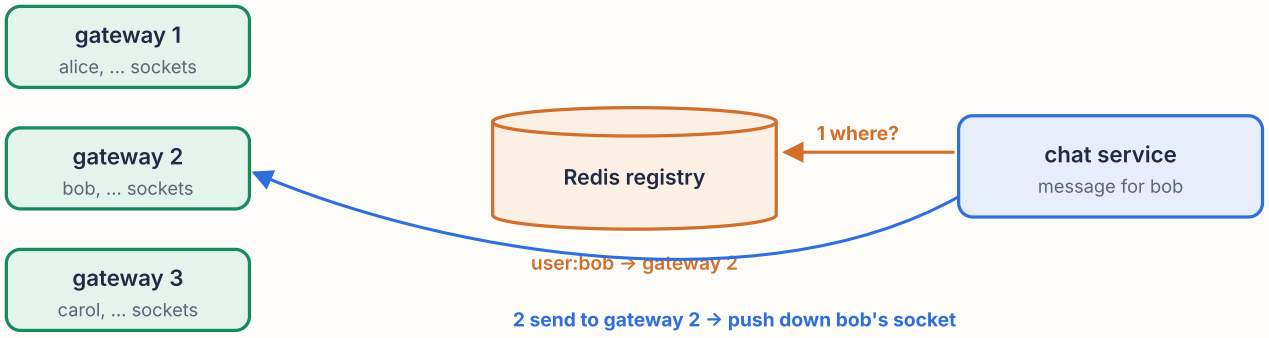


Figure 3. Routing to the owner: a registry says which gateway holds Bob's socket.

5 Keep the state recoverable

- Write important changes to a **durable log or database**, and treat in-memory state as a fast cache of it.
- **Snapshot** periodically, so recovery doesn't replay everything.
- Keep partitions **small and numerous** (hundreds or thousands), so rebalancing moves many small pieces quickly instead of a few giant ones.

COMMON MISTAKES

- Two owners after a network partition, both writing (no fencing).
- As many partitions as nodes, so every scale event moves huge chunks.
- State that exists only in memory, so a crash loses it.

6 Key terms

Partition: a slice of the state, chosen by key

Partition map: which node owns which partition

Fencing token: increasing epoch that rejects old owners

Single owner: only one node handles a partition at a time

Rebalancing: moving partitions when nodes change

Changelog: durable log used to rebuild state

7 Interview questions

Q1. How would you scale a service that must keep in-memory state? BASIC
Partition the state by key, give each partition a single owner using consistent hashing or a coordinator, route requests to the owner, and handle rebalancing by rebuilding, transferring or reconnecting, while keeping the state recoverable from a durable log.

Q2. What's the hardest part? BASIC
Moving ownership safely when nodes join or fail, without two owners and without losing state.

Q3. Design scaling for multiplayer game matches held in memory. SCENARIO
Partition by match ID; a matchmaker assigns each match to one game server and records it; clients connect to that server; state changes are checkpointed so a crashed match can resume elsewhere.

Q4. How do you deliver a chat message to a user connected to one of 50 gateways? SCENARIO
Keep a registry of user → gateway (refreshed by heartbeats), look it up and forward to that gateway; or publish to a per-gateway channel; store messages durably for offline or reconnecting users.

Q5. How do you stop a paused old owner from corrupting state after failover? TRICKY
Use a fencing token: each new owner gets a higher epoch from the coordinator, and storage rejects writes carrying an older epoch.

Q6. Why many small partitions rather than one per node? TRICKY
Moving a partition is the unit of rebalancing; small partitions spread evenly, move quickly and in parallel, and let you add nodes without re-hashing everything.

Next: SD-J-003 · Database Bottlenecks backendroadmap.com